

From Feature-Complete to Production-Ready: A Case Study of Security and Quality Remediation in a Full-Stack Web Application

Aneena Antony

Department of Computer Science and Engineering

Jyothi Engineering College

Thrissur, Kerala, India

Guided by: Prof. Jarin T

Email: aneenaantony321@gmail.com

Abstract—Teams often treat a green build as permission to deploy. We learned the hard way that passing tests and being production-ready are not the same thing. This case study follows an Employee Management System (EMS) we built with React, Node.js/Express, and role-based access control for five organisational roles. All data were synthetic; no real employee records or personally identifiable information were involved. After Phase 1—features working, CI green—a four-hour manual review still turned up five latent problems: a JWT copied into `sessionStorage` beside an `httpOnly` cookie, no real tests on auth/RBAC code, non-transactional file storage, a missing department-scoped access rule, and a refresh endpoint that could not renew expired sessions. Each issue is traced to its cause, mapped to OWASP Top 10 categories, and fixed with version-control evidence. The practical output is a twelve-item pre-deployment checklist other teams can reuse. Our broader conclusion: automated testing is essential, but it verifies what was specified and says little about what was not; a short manual review fills that gap cheaply.

Index Terms—web application security, RBAC, technical debt, code review, OWASP, JWT

I. INTRODUCTION

There is a familiar moment in software projects when someone says the app is done. Features work, tests pass, the pipeline is green. We had exactly that feeling at the end of Phase 1 on our EMS—a React SPA talking to a Node.js REST API with RBAC across Managing Director, Department Manager, Team Lead, Service Engineer, and Employee roles. The system handled employee records, tasks, attendance, leave, expenses, payroll, and management dashboards for a simulated organisation.

What we had not done was ask whether the system was safe to expose. That gap showed up quickly once we read the code with deployment in mind rather than demo scripts. A login test passes whether or not a session token sits in `sessionStorage` where any XSS script could read it. CI stays green when the server-side auth layer has a placeholder test that always succeeds. A test that logs in and checks the dashboard will pass in both cases. These were not textbook examples—they were our codebase.

We paused deployment and spent roughly four hours reviewing authentication, authorisation, persistence, and test adequacy—about 3,500 lines. Five defects emerged (three High, two Medium). Phase 2 remediated all of them. Finding G1 has clean Git tags marking a before-and-after pair; G2–G5 trace to the initial API introduction rather than isolated per-finding branches. The study is deliberately narrow: one codebase, five defect classes appearing together, and a review method others can repeat.

Contributions: (i) a taxonomy of those five co-occurring defects with OWASP mappings; (ii) documented fixes with Git evidence; (iii) a twelve-item production-readiness checklist; and (iv) evidence that a brief manual review catches issues functional tests structurally miss. The paper is organised as follows: Section 2 reviews related work; Section 3 describes the system; Sections 4–6 cover methodology, findings, and remediation; Section 7 discusses validity and concludes.

II. RELATED WORK

A. Technical Debt

Cunningham introduced the debt metaphor while reflecting on the WyCash portfolio system, noting that shipping immature code forces compounding rework [1]. Li et al. mapped 94 technical-debt studies and found measurement and architectural impact dominate the literature [2]. Ernst et al. surveyed practitioners and tied much accumulated debt to deliberate deferral under schedule pressure [3]. We are interested in a narrower subset: defects that stay invisible inside a working build yet carry security or reliability risk once deployed.

B. Web Application Security

OWASP Top 10 [4] catalogues the vulnerability classes most often seen in production web apps. Three mapped onto our findings: A01 broken access control, A03 injection (including XSS), and A07 identification and authentication failures. Hydera et al. reviewed 115 XSS studies and reported that prevention and detection remain central research themes; DOM-based injection via script-accessible storage is still

widely discussed [5]. Gupta and Gupta classify XSS defences and note that exposing tokens in client storage is a common SPA weakness [6]. OWASP’s HTML5 cheat sheet advises against keeping session credentials in `localStorage` or `sessionStorage` because any XSS gains read access [7]. Our angle differs: co-occurring defect classes in one codebase, not prevalence across a population.

C. Code Review

Empirical work shows manual inspection surfaces issues tests miss [8]–[10]. At Microsoft, reviewers valued contextual understanding; security-related changes drew extra scrutiny [8]. McIntosh et al. linked review thoroughness to lower post-release defect density [9]. Google’s review practice caught design and correctness problems linting did not [10]. Moyón et al. discuss integrating security compliance into agile workflows at scale [11]. We contribute a concrete inspection log and a checklist distilled from it. RBAC follows Sandhu et al. [12]; JWT sessions follow RFC 7519 [13]. CWE [14] refines OWASP categories; ASVS [15] lists hundreds of verification requirements—far more than a single pre-deploy gate needs.

III. SYSTEM UNDER STUDY

The EMS covers employee records, tasks, attendance, leave, expenses, payroll, and dashboards for a simulated mid-sized organisation. All data were synthetic; no real employee records or PII were used. The stack is representative of internal business web apps many teams build in-house.

A. Development Context

Work proceeded in two phases. Phase 1 targeted feature completeness: CRUD flows, dashboards, role-specific views, and automated tests that exercised user-visible behaviour. Phase 2 began only after we deliberately halted deployment. That pause was the main methodological choice—treating the passing build as necessary but insufficient evidence of production readiness. The defects reported here were not seeded for the study; they accumulated through incremental delivery, tutorial-influenced auth patterns, and server-side work deferred while the UI matured.

B. Architecture and Access Control

The client is a React 18 SPA (Vite) with React Router and Recharts. The server is a Node.js Express REST API with twelve resource routes covering authentication, employees, tasks, attendance, leave, expenses, payroll, analytics, and supporting endpoints. Phase 1 persistence serialised state to a JSON file; Phase 2 moved to embedded SQLite behind a data-access layer that handles queries and transactions. Security middleware includes Helmet, CORS, login rate limiting, and Zod validation at route boundaries.

Five roles are defined: Managing Director, Department Manager, Team Lead, Service Engineer, and Employee. Demo accounts in Sales and Software departments use Employee and Service Engineer principals respectively. Organisation-wide

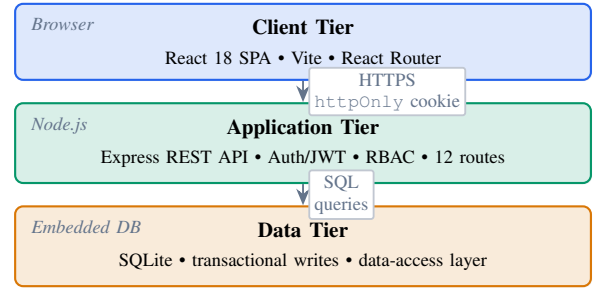


Fig. 1. Post-remediation architecture. Session credentials travel only in an `httpOnly` cookie; JavaScript never reads the JWT.

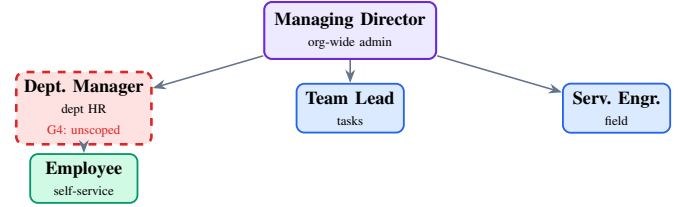


Fig. 2. RBAC role hierarchy. Department Manager (dashed red) lacked department boundary checks before remediation (G4).

admin actions stay with the Managing Director. Department Managers receive department-scoped write access for HR workflows affecting their staff; Team Lead, Service Engineer, and Employee share self-service module access, with task-mutation rules differentiated on the server. Enforcement uses server-side RBAC middleware [12]; client-side role checks only change what the UI shows. Passwords are bcrypt-hashed; sessions use stateless JWTs [13].

Fig. 1 shows the post-remediation layout. Fig. 2 sketches the role hierarchy used for department scoping—the defect in G4 left the department-manager boundary unenforced.

C. Metrics and Phase 1 State

When Phase 1 ended the project had working unit, integration, and E2E tests, a running CI pipeline, and every feature demonstrable. It also had HTTP security headers via Helmet, CORS configuration, login rate limiting, server-side Zod validation on inputs, and bcrypt password hashing. By outward appearance the security basics were in place. Table I lists quantitative characteristics. Backend coverage under 1% was easy to miss because frontend coverage looked healthy at 74%. LOC counts (11,146 total across `src/` and `server/`) were measured with `clloc`, excluding lockfiles; tests ran on Node 20 LTS. By any normal milestone definition—features done, tests green—the EMS was feature-complete. That is precisely the state in which the five latent defects still lived undetected.

Following remediation the system was deployed publicly at <https://p5-employee-management-dashboard.vercel.app>. Source and tagged commits are available at https://github.com/Aneenaanu321/P5_Employee_Management_Dashboard, giving independent evidence that fixes did not break existing behaviour.

TABLE I
SYSTEM METRICS AT END OF EACH PHASE

Metric	Phase 1	Phase 2
Frontend LOC (React/JS)	8,620	8,619
Backend LOC (Node/Express)	2,526	2,524
Total LOC	11,146	11,143
API routes	12	12
Frontend tests	97	97
Backend tests	25	25
Frontend coverage (%)	74	74
Backend coverage (%)	<1	81
Review duration (h)	—	4
Defects found	—	5

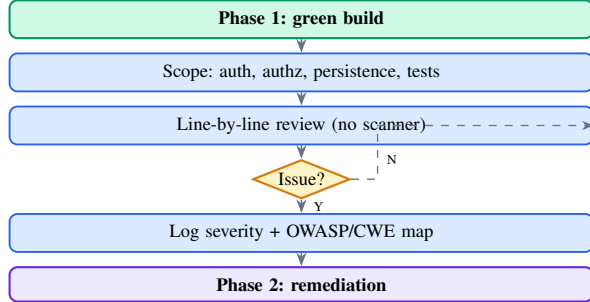


Fig. 3. Review methodology flow from feature-complete build to remediation.

IV. REVIEW METHODOLOGY

Before deployment we carried out a structured manual review scoped to four areas: authentication and session management, server-side authorisation, data persistence, and test-suite adequacy. UI and most business logic were out of scope because existing tests already exercised them. The security-critical backend and test configuration totalled about 3,500 lines.

We read source from known entry points—login route, HTTP client, middleware-decorated routes, persistence layer, CI config—without running an automated scanner. The goal was to reason about intent and design, not pattern-match signatures. Scanners rarely flag two authentication mechanisms coexisting without a canonical choice; careful reading does.

The review took approximately four hours. A finding was recorded when scoped code (i) created a security exposure, (ii) behaved incorrectly relative to obvious intent, or (iii) implemented a security-critical path no test exercised. Severity was assigned as follows: *High* if exploitation required no special precondition beyond normal user access; *Medium* if exploitation required a less-common condition such as a concurrent write race or a non-expired refresh window. OWASP Top 10 and CWE [14] mappings were applied where they fit; G2 and G5 have no precise CWE entry because they are coverage and contract gaps rather than classic code weaknesses. Fig. 3 outlines the process: start from a green Phase 1 build, walk the four risk areas, record each issue with symptom and root cause, then enter Phase 2 remediation once all areas are covered.

V. FINDINGS

The review produced five findings (Table II). G1, G2, and G4 are High severity—exploitable under normal authenticated use. G3 and G5 are Medium: G3 needs concurrent writes; G5 needs a session to actually expire, which our integration tests avoided with long-lived tokens.

A. G1: JWT Duplicated in sessionStorage

Symptom. The server issued the session token as an `httpOnly` cookie, which JavaScript cannot read. Unfortunately the client also stored the same token in `sessionStorage` (plus an in-memory copy) and attached `Authorization: Bearer` on every request. Both paths were live.

Root cause. Two authentication strategies were implemented in parallel during development. The cookie approach won, but the `sessionStorage` logic was never removed.

Risk. Because `sessionStorage` is readable by any script on the page, one XSS flaw anywhere would expose the session. The `httpOnly` cookie was meant to prevent that, but an identical copy in storage made the protection ineffective. The real posture was set by the weaker mechanism (OWASP A03/A07; CWE-922).

B. G2: No Tests on Auth or RBAC

Symptom. The server test step in CI executed a single assertion that was always true. Login, JWT signing and verification, and every RBAC middleware function had zero automated coverage.

Root cause. Test effort concentrated on the frontend, where behaviour is visible. The server-side security logic was left uncovered; the placeholder kept the pipeline green.

Risk. Any regression in login, token verification, or access control would sail through CI undetected. A team treating a green build as proof that authentication works had no basis for that confidence (nearest OWASP A04).

C. G3: Non-Transactional File Store

Symptom. Application state lived in memory and was written by serialising the entire dataset to JSON on each update. There was no record-level update mechanism.

Root cause. A flat JSON file was reasonable for an early prototype. The system grew, but the persistence layer was never revisited.

Risk. Whole-file writes are not atomic—concurrent requests can interleave and lose updates; a crash mid-write can corrupt the file. Neither failure throws a clean application error; both are plausible under modest production concurrency (CWE-362).

D. G4: Missing Department-Scoped Rule

Symptom. The `requireOwnEmployeeId` middleware should restrict Department Manager writes to employees in their own department. At review time it enforced self-only access but never compared the target employee’s department to the manager’s.

TABLE II
FINDINGS IN THE FEATURE-COMPLETE SYSTEM

ID	Issue	Sev.	OWASP	CWE
G1	JWT in <code>sessionStorage</code>	H	A03, A07	922
G2	Auth/RBAC untested	H	A04	—
G3	Non-transactional store	M	A04	362
G4	Missing dept. scope	H	A01	862
G5	Refresh needs valid token	M	A07	—

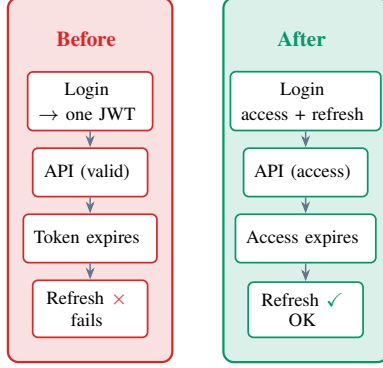


Fig. 4. G5 token lifecycle before (left) and after (right) remediation.

Root cause. The rule was deferred under schedule pressure and never completed. No test asserted its presence.

Risk. An unimplemented authorisation branch may silently over-permit (privilege escalation) or over-deny. Either failure stays invisible without targeted negative-case tests (A01; CWE-862). Listing 7 shows the missing branch at review time.

E. G5: Refresh Endpoint Could Not Refresh

Symptom. The API exposed a token-refresh route, but presenting an expired token returned an authentication error; sessions could not be renewed after expiry.

Root cause. A single JWT served as both access and refresh credential. The refresh route required a valid, non-expired token, so access-token expiry simultaneously invalidated the only credential that could trigger renewal.

Risk. Users were silently logged out at session expiry with no recovery path. The endpoint advertised a contract it could not honour—a condition integration tests never exercised because tokens were configured not to expire (A07). Fig. 4 contrasts the broken single-token design with the corrected dual-token model.

F. Co-Occurrence Pattern

None of the five findings existed in isolation. G1 (exposed token) and G2 (untested auth) compounded each other: even after removing `sessionStorage`, regressions could reintroduce client-side storage without CI noticing. G4 (missing authz branch) and G2 shared the same root—deferred server work without failing tests. G3 (file store) and G5 (refresh contract) both reflected prototype choices that survived feature growth. Treating them as a set motivated a checklist that spans storage, session, authz, and test hygiene rather than a single-issue hotfix list.

TABLE III
BEFORE-AND-AFTER REMEDIATION SUMMARY

ID	Before	After
G1	Token in <code>sessionStorage</code> + Bearer	<code>httpOnly</code> cookie only
G2	Placeholder server test	Integration + middleware suites
G3	Whole-file JSON rewrite	SQLite with transactions
G4	Dept. branch missing	Rule + allow/deny tests
G5	Single JWT	Separate access + refresh tokens

```

function cookieBase() {
  const isProd = process.env.NODE_ENV === 'production';
  return {
    httpOnly: true,
    secure: isProd,
    sameSite: 'lax',
  };
}

```

Fig. 5. G1: server cookie configuration after remediation.

VI. REMEDIATION

Phase 2 addressed every finding. Table III summarises before-and-after states; subsections below give implementation detail.

A. Fixes

For G1 we deleted all client-side token handling. Listing 6 shows the client change; Listing 5 shows server cookie settings after remediation. Fetch calls now use `credentials: 'include'` so the browser sends the `httpOnly` cookie without JavaScript touching the secret. `sameSite: 'lax'` limits cross-origin cookie use on state-changing requests and addresses the residual CSRF risk once the Bearer header was gone. Deployment routes API traffic same-origin via host rewrites, so `Lax` suffices without `SameSite=None`.

For G2 we added an integration suite over auth endpoints and representative routes, plus unit tests on each RBAC rule with allowed and denied roles. The placeholder assertion was removed; a broken auth path now fails CI. Backend coverage climbed from under 1% to 81%. Middleware tests exercise both success and 403 denial paths so regressions in role enforcement are caught before merge.

G3 replaced the JSON store with SQLite and a relational data-access layer whose transactions make writes atomic. G4 completed the department-manager branch in `requireOwnEmployeeId`: when the caller is a Department Manager and the target employee ID differs from their own, the middleware loads the employee record and compares departments before allowing the operation. Two tests lock in the boundary—same-department allow, cross-department deny. G5 split credentials into a short-lived access JWT (minutes) and a long-lived refresh JWT accepted only by the refresh route—so expiry of the access token no longer blocks renewal while the refresh token remains valid.

B. Version-Control Evidence

Tag `phase-1-complete` (262c924) marks the last commit where the client still duplicated the JWT

```
// BEFORE: token in sessionStorage (XSS-readable)
const t = sessionStorage.getItem('access_session');
fetch(url, { headers: { Authorization: 'Bearer ' + t }});

// AFTER: httpOnly cookie only
fetch(url, { credentials: 'include' });
```

Fig. 6. G1 client auth handling before and after remediation.

```
// G4 BEFORE: dept-manager branch absent
if (role === DEPT_MANAGER && bodyId &&
    bodyId !== workforceId) {
  // branch never implemented
}
if (bodyId && bodyId !== workforceId) {
  return deny('Can only act on own record');
}
```

Fig. 7. G4: requireOwnEmployeeId before remediation (excerpt).

TABLE IV
PRODUCTION-READINESS REVIEW CHECKLIST

#	Item
1	No credential in localStorage/sessionStorage.
2	Session uses httpOnly cookies; CSRF protection present.
3	Auth code has dedicated automated tests.
4	Each access rule has negative (deny) tests per role.
5	Authorisation enforced on server; client checks are UX only.
6	CI has no placeholder or always-true tests.
7	No dead branches in security middleware.
8	Data store provides atomic writes/transactions.
9	Concurrent-write behaviour tested or documented.
10	Token lifecycle works for expired sessions.
11	No hard-coded secrets; fail closed if secrets missing.
12	External input validated and size-limited at server.

in sessionStorage. Tag phase-2-complete (d184659) is the G1 remediation commit. `git diff phase-1-complete phase-2-complete` yields a G1-only delta across seven files (auth client, cookie handling, related tests) without unrelated UI commits that appear if older tips such as 8fee696 are used as the Phase 1 marker. Findings G2–G5 were bundled when the REST API was introduced (dded48e..df63307). Named branches that once pointed at the same commit were removed from the repository because they falsely implied per-finding isolation. Reviewers can run `git show d184659` for G1 or `git diff dded48e df63307` for the server-side bundle. Independent reviewers reproduced metrics using `cloc src server` and the project’s test runners on Node 20 LTS; native modules such as `better-sqlite3` may require a rebuild on newer Node versions.

C. Production-Readiness Checklist

Looking across the five findings, all sat in areas functional testing tends to skip. We generalised them into the twelve-item gate in Table IV. Items 1–10 map directly to G1–G5; items 11–12 cover hard-coded secrets and unvalidated input—standard deployment concerns any checklist would need.

Compared with OWASP ASVS [15], our checklist is empirically derived, scoped to a deployment gate, and fits one table. ASVS organises several hundred requirements across assurance levels; that depth supports full-lifecycle programmes but is heavy for a team about to flip a deploy switch. Items 1–

2 align with ASVS session management (V3); item 4 with access control (V4); items 3, 6, and 7 with architecture and verification discipline (V1). Teams targeting higher assurance should treat Table IV as a first pass, then consult ASVS for depth. For internal HR-style apps like ours, items 1–6 alone would have caught four of the five findings before deploy.

D. Applying the Checklist in Practice

We used the checklist retrospectively: each item maps to at least one finding we missed. Item 1 would have flagged G1 immediately; items 3 and 6 would have exposed G2; item 7 would have prompted inspection of unfinished middleware (G4); items 8–9 cover G3; item 10 covers G5. For a team about to deploy, we suggest walking the table in order, ticking items only with evidence (test name, code pointer, or config snippet). Items failing the gate should block release or be explicitly accepted as documented risk. Converting items 3, 4, 6, and 10 into CI assertions would automate much of the regression protection we added manually in Phase 2.

VII. DISCUSSION

A. Why Tests Missed These Defects

Four findings involve properties standard functional tests do not assert. Token placement does not change login success. JSON atomicity does not break single-user happy paths. Refresh behaviour stays hidden when tokens never expire in CI. G4 is functional, but nobody wrote a negative authorisation test for cross-department access.

The underlying pattern is simple: tests verify what someone thought to specify. Review asks what the system must refuse to do and checks whether the code actually refuses it. The two approaches complement each other; neither replaces the other. In our case, frontend E2E tests gave high confidence in user journeys while leaving the server-side trust boundary effectively unverified.

B. Generalisability and Validity

This is a single-project case study. We do not claim statistical generalisation. The mitigating argument is that each defect class appears often in incrementally built SPAs—script-accessible tokens, uneven test focus, persistence that outlives prototypes, stubbed auth branches, conflated refresh semantics—and the checklist states generic verifiable properties rather than EMS-specific quirks. Storing session tokens in sessionStorage is a pattern OWASP explicitly cautions against [7]. Concentrating coverage on visible UI behaviour while deferring server security is a predictable outcome of feature-driven delivery.

Internal validity is limited by one reviewer and no parallel scanner run. A second reviewer or tools like Semgrep or OWASP ZAP might surface additional issues. The EMS was built as a research artefact; defects arose naturally during incremental development but readers should weigh that context. Construct validity depends on pragmatic definitions: “feature-complete” means working features plus passing tests;

“production-ready” is operationalised by the checklist in Table IV.

Regarding cost: four hours on 3,500 lines surfaced three High and two Medium issues. A formal penetration test of similar scope would typically take days and cost more. Teams under schedule pressure may still prefer this lightweight gate before paying for external assessment.

C. Lessons for Practitioners

Four lessons stood out for us. First, green CI can be actively misleading when placeholder tests exist—treat backend coverage on security modules as a first-class metric, not an afterthought. Second, SPA tutorials often show Bearer tokens in browser storage; teams should pick one session strategy and delete the other code path, not layer both. Third, negative tests for authorisation are as important as happy-path feature tests; a rule deferred without a failing test is a rule that does not exist yet. Fourth, prototype persistence and auth shortcuts should carry explicit expiry criteria—when user count, write frequency, or session lifetime requirements change, revisit the layers that were “good enough” for the demo.

VIII. CONCLUSION

We set out to document what happens in the gap between a feature-complete build and a genuinely production-ready deployment. Five security and quality defects—three High severity, two Medium—survived the entire development process and a passing automated test suite. They persisted not because the tests were badly written, but because they checked functional behaviour, and these defects live outside functional behaviour.

The fixes were not complicated once identified. Removing `sessionStorage` usage, adding real server tests, migrating to SQLite, completing one middleware branch, and splitting access/refresh tokens were all incremental changes. The review that found them covered the security-critical portion of the codebase in four hours. The checklist we derived fits on one table. None of this required specialist tooling or a paid audit—only the decision to look at the parts of the codebase tests tend not to reach before shipping.

A green build means your tests pass, not that your application is ready for production. Lightweight, targeted code review is a cheap and effective way to close that gap. For teams building similar full-stack internal apps, we recommend running Table IV as a final gate even when CI is green.

Future work includes running automated scanners (Semgrep, OWASP ZAP) in parallel and comparing coverage against manual findings; applying the checklist to additional open-source EMS-like codebases; engaging an independent second reviewer; and turning selected checklist items into automated CI rules.

ETHICAL CONSIDERATIONS

The case study uses a synthetic Employee Management System with no real employee records or personally identifiable information. Security defects were identified through

manual review of source code in a controlled development environment; no third-party systems were probed or exploited. Remediation was applied before public deployment. The artifact repository documents vulnerabilities and fixes for reproducibility; readers should not treat disclosed patterns as invitations to attack unrelated live systems. Public URLs are provided so others can verify that remediation preserved functionality, not to encourage unauthorised access attempts.

ACKNOWLEDGMENT

The author thanks Prof. Jarin T, Jyothi Engineering College, for guidance and review of this work.

REFERENCES

- [1] W. Cunningham, “The WyCash portfolio management system,” in *Addendum to the Proc. OOPSLA*. ACM, 1992, pp. 29–30.
- [2] Z. Li, P. Aygeriou, and P. Liang, “A systematic mapping study on technical debt and its management,” *J. Syst. Softw.*, vol. 101, pp. 193–220, 2015.
- [3] N. A. Ernst *et al.*, “Measure it? manage it? ignore it? software practitioners and technical debt,” in *Proc. ESEC/FSE*. ACM, 2015, pp. 50–60.
- [4] OWASP Foundation, “OWASP Top 10:2021,” <https://owasp.org/Top10/2021>.
- [5] I. Hydar *et al.*, “Current state of research on cross-site scripting (XSS): A systematic literature review,” *Inf. Softw. Technol.*, vol. 58, pp. 170–186, 2015.
- [6] S. Gupta and B. B. Gupta, “Cross-site scripting (XSS) attacks and defense mechanisms,” *Int. J. Syst. Assurance Eng. Manag.*, vol. 8, no. 1, pp. 512–530, 2017.
- [7] OWASP Foundation, “HTML5 security cheat sheet,” https://cheatsheetseries.owasp.org/cheatsheets/HTML5_Security_Cheat_Sheet.html, 2024.
- [8] A. Bacchelli and C. Bird, “Expectations, outcomes, and challenges of modern code review,” in *Proc. ICSE*. IEEE, 2013, pp. 712–721.
- [9] S. McIntosh *et al.*, “An empirical study of the impact of modern code review practices on software quality,” *Empirical Softw. Eng.*, vol. 21, no. 5, pp. 2146–2189, 2016.
- [10] C. Sadowski *et al.*, “Modern code review: A case study at Google,” in *Proc. ICSE-SEIP*. ACM, 2018, pp. 181–190.
- [11] F. Moyón *et al.*, “How to integrate security compliance requirements with agile software engineering at scale?” in *Proc. PROFES*, LNCS vol. 12562. Springer, 2020, pp. 69–87.
- [12] R. S. Sandhu *et al.*, “Role-based access control models,” *Computer*, vol. 29, no. 2, pp. 38–47, 1996.
- [13] M. B. Jones, J. Bradley, and N. Sakimura, “JSON web token (JWT),” IETF RFC 7519, May 2015.
- [14] MITRE Corporation, “Common weakness enumeration (CWE),” <https://cwe.mitre.org/>, 2024.
- [15] OWASP Foundation, “Application security verification standard 4.0,” <https://owasp.org/www-project-application-security-verification-standard/>, 2021.